Full-stack dApp: web page + wallet + contract

Day 13 of 15 · 27 October 2026 · Phase: Building real applications

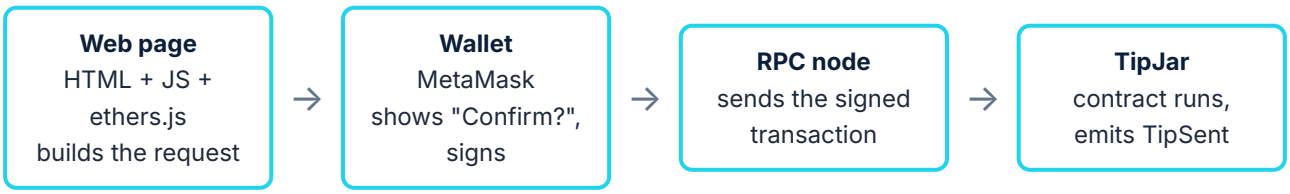
Study guide by Manish Khadka

1. Learning objectives

- Explain the three layers of a dApp (page, wallet, chain) and who holds the private key.
- Build a contract the UI can use safely: bounded reads, clear errors, events.
- Show untrusted on-chain text without XSS; handle wrong network, cancelled signing and stale config.
- Deploy to a local chain or Sepolia while keeping secrets out of git, and test a frontend in a real browser.

2. The concept, step by step

A **dApp** is an ordinary website whose back end is a smart contract. The page never sees a private key: it *asks* the wallet, the wallet *asks you*, and only then a signed transaction goes to the chain.



1. Click *Connect*: the page calls `eth_requestAccounts`; the wallet asks permission and returns the address.
2. The page **reads** (free, instant): `tipCount()`, `getTips(0,20)`.
3. You type an amount and a message. The page checks them **first** (fast, kind errors).
4. The page asks the wallet to send `tip(message)` with value. You confirm; the wallet signs.
5. The page waits for the receipt (about 12 s on Sepolia) and refreshes the list.

	Reading (view)	Writing (transaction)
Gas	No	Yes
Signature	No	Yes (wallet popup)
Speed	Instant	Waits for a block

Network	Chain id	ETH is	Use for
Hardhat local	31337	fake, 10,000 per account	building, tests
Sepolia	11155111	free from a faucet	rehearsal
Mainnet	1	real money	production

3. Key code walkthrough

The contract: bounded, explicit, safe withdrawal

```
function tip(string calldata message) external payable {
    if (msg.value < MIN_TIP) revert TipTooSmall(msg.value, MIN_TIP);   // spam costs money
    uint256 len = bytes(message).length;                               // BYTES, not characters
    if (len == 0) revert EmptyMessage();
    if (len > MAX_MESSAGE_BYTES) revert MessageTooLong(len, 140);
    ...push to _tips, totalTipped += msg.value, emit TipSent(...)
}
function withdraw(address payable to) external onlyOwner {
    uint256 amount = address(this).balance;       // live balance: a re-entrant call finds 0
    if (amount == 0) revert NothingToWithdraw();
    emit Withdrawn(to, amount);                   // effects before the external call
    (bool ok, ) = to.call{value: amount}("");
    if (!ok) revert TransferFailed();
}
```

`getTips(offset, limit)` returns the newest tips first and caps a page at 50, so no read can grow without limit (Day 09 DoS rule).

The page: untrusted text is text

```
el.textContent = tip.message;      // never innerHTML: a stranger wrote this message
// + Content-Security-Policy without 'unsafe-inline'  (second lock)
// + ESLint rule and a test that ban innerHTML in frontend/
```

The secrets check (where the evening audit found bugs)

```
const HARDHAT_PUBLIC_KEYS = Array.from({length: 20}, (_, i) =>
  HDNodeWallet.fromPhrase(PUBLIC_MNEMONIC, undefined, `m/44'/60'/0'/0/${i}
`).privateKey.toLowerCase());
if (BigInt(key) === 0n || BigInt(key) >= CURVE_N) problems.push("not a valid private key");
else if (HARDHAT_PUBLIC_KEYS.includes(key.toLowerCase())) problems.push("PUBLIC test key");
```

4. Results from the run

Measure	Result
Deploy gas (TipJar)	665,179
Tips of 28, 14 and 21 characters	136,722 / 102,390 / 147,360 gas (average 128,824)
Byte sizes of those messages	28 / 17 / 33 bytes (emoji and Nepali text cost more bytes than characters)
Owner withdraw of 0.008 ETH	30,214 gas
Tests	45 passing: 13 contract, 2 fuzz, 13 deploy/env/server, 11 browser E2E, 3 evening audit, 3 exercises

The first tip costs more than the second because it writes a fresh storage slot (`totalTipped` from 0, about 20,000 gas). The third is dearest because its message is longest in bytes.

5. QA and security findings

ID	Sev.	What was wrong	Fix
L1	Low	Two quick account switches left two polling timers	Generation number; older setup stops
L2	Low	A damaged config file was silently replaced	Refuse to overwrite unparseable config
L3	Low	Deploy to the in-memory chain "succeeded" and wrote a dead address	Refuse the `hardhat` network
L4	Low	Page could be framed (clickjacking)	X-Frame-Options and frame-ancestors headers
E1	Low	Only 1 of 20 public Hardhat keys was blocked for Sepolia	All 20 derived and refused
E2	Low	All-zero private key accepted	Valid range 1 to curve order minus 1
E3	Low	"https://" with no host accepted	Parse with `new URL()`, need a host

Re-entrancy on `withdraw`, huge page requests, emoji at the 140-byte limit, path traversal and XSS payloads were all attacked and held. No high or medium findings; Slither shows 6 informational results only.

6. Common mistakes and interview questions

- Putting a private key in the frontend or committing `.env`.

- Rendering chain data with `innerHTML`.
- Counting message length in characters when the contract counts bytes.
- Not handling `chainChanged` / `accountsChanged`, or trusting a stale contract address.

1. **Why can't the page sign by itself?** It never holds the key; the wallet signs so the user approves every transaction.
2. **View call vs transaction?** A view call is a free read answered by a node; a transaction changes state, needs a signature and gas, and waits for a block.
3. **Why validate in both page and contract?** The page gives fast, kind errors; the contract is the real defence because anyone can skip the page.
4. **How do you stop XSS from on-chain text?** Render with `textContent`, add a CSP without inline scripts, and lint against `innerHTML`.
5. **Why paginate `getTips`?** An unbounded array read can exceed gas/RPC limits; a page cap keeps every call bounded.

7. Exercises summary

1. **Easy:** `fits()` and `truncateToFit()`: bytes vs characters, checked against the contract.
2. **Medium:** `fetchAllTips()` using only paged reads (0, 1, 50, 51, 120 tips).
3. **Challenge:** a leaderboard from `TipSent` events whose sum equals `totalTipped()`.

Run with `npm run ex` in `day-13`.

8. Cheat sheet

dApp = page + wallet + contract. Page builds, wallet signs, chain runs.
Reads are free; writes cost gas and wait.
On-chain data is user input: `textContent` + CSP.
Validate twice: page for kindness, contract for safety.
Secrets: `.env` (git-ignored); commit only `.env.example`; block all public test keys.
Handle: chainChanged, accountsChanged, no wallet, Cancel (code 4001), no code at address.
Commands: `npm start` · `npm test` · `npm run node` · `npm run deploy:local` · `npm run check:env`

9. References

- ethers.js v6 docs: https://docs.ethers.org/v6/getting-started/
- EIP-1193 Provider API: https://eips.ethereum.org/EIPS/eip-1193
- MetaMask connect docs: https://docs.metamask.io/wallet/how-to/connect/
- Hardhat deploying guide: https://hardhat.org/hardhat-runner/docs/guides/deploying

- ethereum.org networks: https://ethereum.org/en/developers/docs/networks/
- OWASP XSS prevention: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- Solidity docs: https://docs.soliditylang.org/en/v0.8.24/
- SWC registry (reentrancy SWC-107): https://swcregistry.io/docs/SWC-107